

untangle::async

Generated by Doxygen 1.18.0

1	async	1
2	Topic Index	7
2.1	Topics	7
3	Namespace Index	9
3.1	Namespace List	9
4	Class Index	11
4.1	Class List	11
5	File Index	13
5.1	File List	13
6	Topic Documentation	15
6.1	namespace untangle: functions	15
7	Namespace Documentation	17
7.1	untangle Namespace Reference	17
7.1.1	Detailed Description	17
8	Class Documentation	19
8.1	untangle::async::attachment Struct Reference	19
8.1.1	Detailed Description	19
8.2	untangle::async::execution< actionT > Class Template Reference	19
8.2.1	Detailed Description	21
8.2.2	Constructor & Destructor Documentation	21
8.2.2.1	execution() [1/2]	21
8.2.2.2	execution() [2/2]	22
8.2.2.3	~execution()	22
8.2.3	Member Function Documentation	22
8.2.3.1	create_instance()	22
8.2.3.2	bind_action_and_method()	23
8.2.3.3	bind_action_and_function()	23
8.2.3.4	is_running()	24
8.2.3.5	is_busy()	24
8.2.3.6	run()	25
8.2.3.7	start()	25
8.2.3.8	stop()	25
8.2.3.9	add_action()	26
8.2.3.10	add_task()	27
8.2.3.11	attach()	28
8.2.3.12	detach()	28
8.2.3.13	results()	29
8.2.4	Member Data Documentation	29
8.2.4.1	action_execute	29

8.2.4.2	action_stop	29
8.2.4.3	action_is_running	30
8.2.4.4	on_finished	30
8.2.4.5	on_error	30
8.3	untangle::async::execution_poll Class Reference	31
8.3.1	Detailed Description	31
8.3.2	Constructor & Destructor Documentation	31
8.3.2.1	~execution_poll()	31
8.3.3	Member Function Documentation	32
8.3.3.1	add()	32
8.3.3.2	remove()	32
8.3.3.3	withdraw()	32
8.3.3.4	is_running()	33
8.4	untangle::async::invalid_attachment Struct Reference	33
8.4.1	Detailed Description	33
8.4.2	Constructor & Destructor Documentation	33
8.4.2.1	invalid_attachment()	33
8.4.3	Member Function Documentation	34
8.4.3.1	what()	34
9	File Documentation	35
9.1	async.hpp	35
	Index	43

async

The interface consists of one header file that exposes a generic execution class. It is a wrapper of a queue of callables that are invoked on other thread. The callables are pumped into the queue by invoking an action from the caller thread.

There are provided two execution modes: one-off and continuous.

C++23. The interface reports warnings with `std::println`, so it needs a standard library that provides `<print>` — GCC 14 or newer; older `libc++` and MSVC releases may not have it yet.

```
#include <async.hpp>

void f()
{
    std::cout << "f thread " << std::this_thread::get_id() << std::endl;
}

int main()
{
    std::cout << "main thread " << std::this_thread::get_id() << std::endl;

    // declare an execution specialized for an action type; it must be held by a shared pointer,
    // because an async binding keeps only weak ownership of it
    // the name is optional, and identifies the execution in the warnings it reports
    using void_execution = untangle::async::execution<std::function<void(void)>;
    auto execution = void_execution::create_instance<"oneoff">;
```

```

std::function<void(void)> action;
// create async binding between the action and f; the execution is passed as a shared pointer,
// which the binding holds weakly - that is what lets the action outlive it safely
void_execution::bind_action_and_function(action, f, execution);
// whenever the action is invoked, a new callable wrapping f will be added to the execution's action queue
action();
// run the execution
execution->run();

// add the execution object to a poll of one's own; the poll answers for what was added to it
untangle::async::execution_poll poll;
poll.add(*execution);

// wait the polled executions to finish
while(poll.is_running())
{
    std::this_thread::sleep_for(std::chrono::milliseconds(1));
}

return 0;
}

```

example: one-off execution of a class method

```

#include <async.hpp>

class A
{
public:
    void f(int x)
    {
        std::cout << "A::f " << x << " thread " << std::this_thread::get_id() << std::endl;
    }
    std::function<void(int)> action;
};

int main()
{
    std::cout << "main thread " << std::this_thread::get_id() << std::endl;

    auto a = std::make_shared<A>();

    // declare an execution specialized for an action type; it must be held by a shared pointer
    using int_execution = untangle::async::execution<std::function<void(int)>>;
    auto execution = int_execution::create_instance("oneoff_method");
    // note both the bound object and the execution must be shared pointers
    int_execution::bind_action_and_method(a->action, a, &A::f, execution);
    // whenever the action is invoked, a new callable wrapping f will be added to the execution's action queue
    a->action(10);
    // run the execution
    execution->run();

    // add the execution object to a poll of one's own; the poll answers for what was added to it
    untangle::async::execution_poll poll;
    poll.add(*execution);

    // wait the polled executions to finish
    while(poll.is_running())
    {
        std::this_thread::sleep_for(std::chrono::milliseconds(1));
    }

    return 0;
}

```

continuous execution



example: continuous execution of a plain function

```

#include <async.hpp>

void f()
{

```

```

    std::cout << "f thread " << std::this_thread::get_id() << std::endl;
}

int main()
{
    std::cout << "main thread " << std::this_thread::get_id() << std::endl;

    // declare an execution specialized for an action type; it must be held by a shared pointer
    using void_execution = untangle::async::execution<std::function<void(void)>>;
    auto execution = void_execution::create_instance("continuous");
    std::function<void(void)> action;
    // create async binding between the action and f
    void_execution::bind_action_and_function(action, f, execution);

    // run the execution
    execution->start();
    // add f to the execution's queue (three times)
    action();action();action();
    // stop the execution
    execution->stop();

    // add the execution object to a poll of one's own; the poll answers for what was added to it
    untangle::async::execution_poll poll;
    poll.add(*execution);

    // wait the polled executions to finish
    while(poll.is_running())
    {
        std::this_thread::sleep_for(std::chrono::milliseconds(1));
    }

    return 0;
}

```

queueing and lifetime

Queueing is thread safe. An action may be invoked from any thread; the execution's list is guarded, and the worker is woken rather than polled for.

`stop()` ends the execution's working life. The worker runs whatever was queued before the call and then leaves. An action invoked after `stop()` is refused rather than queued, because it would never run, and the refusal is reported:

```
warning: execution 'continuous' is stopped, action not added
```

A task is refused after `stop()` for the same reason, and says so in its own words:

```
warning: execution 'continuous' is stopped, task not added
```

A task is also refused before it is ever queued if it could not do its job — no callback to notify, or nothing to run. That one is the caller's own mistake and is caught while they are still on the stack, rather than surfacing later as a failure of a task whose action had in fact succeeded:

```
warning: execution 'continuous' refused a task that cannot report
```

An action bound to an object that has since been destroyed is dropped the same way, rather than ending the process:

```
warning: execution 'continuous' dropped an invalid action: bind: invalid object
```

The worker is detached and cannot be joined. Waiting for an execution therefore means waiting on its running state, which is what `execution_poll` reports and what the examples above do. The destructor waits on the same state, so an execution that goes out of scope while its worker is still running blocks until the worker is finished rather than leaving it reading freed memory.

The poll holds a pointer to each execution added to it, so an execution withdraws itself in its destructor - from every poll it was added to, since it may be in more than one. A poll destroyed first likewise releases what it still holds, so a poll may be a local. `execution_poll::remove()` is available for withdrawing one earlier; calling it is not required.

tasks: work that reports back

Everything above queues actions, through `add_action()`. An action is fire and forget: it runs, what it returns is dropped, and the caller hears nothing more about it. A task, through `add_task()`, carries the callback it must notify:

```
auto exec = untangle::async::execution<std::function<int(int)>>::create_instance("scale");
exec->start();

// the callback comes last, after the arguments the action is bound to
exec->add_task([](int n) { return n * 2; }, 21, [](int result) { /* result == 42 */ });
```

A task whose action returns void still reports that it finished, with a callback taking nothing — finished is the message and the result is optional:

```
exec->add_task([](int by) { /* ... */ }, 5, [] { /* finished */ });
```

The callback is the last argument, and the signature cannot say so. A parameter pack cannot be followed by a deducible parameter, so it arrives inside the arguments and `untangle::bind_task()` splits it off. It must be callable with the action's result and return nothing — or callable with nothing at all, when the action returns void. A missing or unusable one is a compile error rather than silence.

Why a task carries its own callback rather than being handed one. The actuator has a callback convention already: pass a trailing callable to an actuator's call operator and it is invoked with each action's result. That convention reads the trailing argument of an invocation, and a queued call has no invocation left to read from — so the callback has to travel with the work. That is what a task is.

`add_task()` answers bool, and the answer is the whole report. It refuses a task the execution can never run — because it is stopped — or one that could never do its job: no callback to notify, or nothing to run. A refused task is dropped, so it never runs and never notifies, and a caller waiting on the callback alone would wait for ever.

Finished does not mean failed. A task whose action throws is not notified: there is no result to hand over, and for a void task no completion to report either. What it threw goes to `on_error`, as a failing action's does. And because a callback runs inside the same try as the task that owns it, `on_error` can fire for a task whose action in fact succeeded — the work was done and only the telling failed.

`on_finished` is not a task's callback. It is raised once per pass, when the queue has drained, and a pass that ran only tasks raises it like any other. A task's own callback has already run by then, inside the same drain and on the same thread. Two notifications, two meanings: one per task, one per drained queue.

Both kinds share one queue and one worker. Within a single pass every action runs before any task, so the two do not interleave in the order they were queued; across passes the queue is still first in, first out.

building the tests

```
cmake -S test -B test/build
cmake --build test/build
ctest --test-dir test/build
```

`test/CMakeLists.txt` fetches googletest at configure time, so the first configure needs network access. The actuator submodule has to be present: `git submodule update --init`.

sanitizers

ASYNC_SANITIZE builds the tests under a sanitizer. It is off by default, because a sanitized build is several times slower and ThreadSanitizer does not ship for every toolchain.

```
cmake -S test -B test/build-asan -DASYNC_SANITIZE=address
cmake --build test/build-asan
ctest --test-dir test/build-asan
```

Accepted values are address, thread, undefined, or empty. Anything else is refused at configure time rather than passed through to the compiler.

Use a separate build directory per sanitizer. address and thread instrument the same accesses in incompatible ways and cannot be combined, so one build directory is one sanitizer.

The two answer different questions. ThreadSanitizer names both sides of a data race, which is how the queue's races were diagnosed. AddressSanitizer reports a use-after-free, which is how the dangling attachment was. Neither sees everything: a defect reached through a destroyed `std::mutex` throws `std::system_error` from inside `libsystem` before any instrumented load runs, so both sanitizers stay silent. Reach for a direct instrumented read when that happens.

Chapter 2

Topic Index

2.1 Topics

Here is a list of all topics with brief descriptions:

namespace untangle: functions [15](#)

Chapter 3

Namespace Index

3.1 Namespace List

Here is a list of all documented namespaces with brief descriptions:

untangle	Interface to untangle::async::execution class	17
--------------------------	---	--------------------

Chapter 4

Class Index

4.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

untangle::async::attachment	
One execution's place in the attachment graph: who attached it, if anyone	19
untangle::async::execution< actionT >	
Asynchronous template execution class	19
untangle::async::execution_poll	
Poll that reports whether any execution added to it is still running	31
untangle::async::invalid_attachment	
Raised by execution::attach() when the attachment asked for is refused	33

Chapter 5

File Index

5.1 File List

Here is a list of all documented files with brief descriptions:

async.hpp		35
---------------------------	-------	----

Chapter 6

Topic Documentation

6.1 namespace untangle: functions

Chapter 7

Namespace Documentation

7.1 untangle Namespace Reference

Interface to [untangle::async::execution](#) class.

7.1.1 Detailed Description

Interface to [untangle::async::execution](#) class.

Chapter 8

Class Documentation

8.1 `untangle::async::attachment` Struct Reference

One execution's place in the attachment graph: who attached it, if anyone.

```
#include <async.hpp>
```

Public Attributes

- `std::weak_ptr< attachment >` `attacher`
The link of the execution that attached this one; empty when nothing has.

8.1.1 Detailed Description

One execution's place in the attachment graph: who attached it, if anyone.

At namespace scope, so it is one type rather than one per specialisation: an execution may be attached by an execution of any specialisation, and the chain is walked without knowing any link's action type. It dies with its execution, so it doubles as a liveness token - a link that still locks belongs to an execution that is still there.

The documentation for this struct was generated from the following file:

- `async.hpp`

8.2 `untangle::async::execution< actionT >` Class Template Reference

Asynchronous template execution class.

```
#include <async.hpp>
```

Public Member Functions

- [execution](#) ()
Constructs an execution with the default name. Call [create_instance\(\)](#) instead.
- [execution](#) (std::string exec_name)
Constructs a named execution. Use [create_instance\(\)](#) for one that is to be bound.
- [~execution](#) ()
Destroys the execution, once its worker has left.
- bool [is_running](#) () const
Is a worker of this execution's still there?
- bool [is_busy](#) () const
Is this execution working through its action queue?
- void [run](#) ()
Runs what is queued on a worker of its own, once, and keeps the results.
- void [start](#) ()
Starts a continuous worker that runs whatever is queued until [stop\(\)](#).
- void [stop](#) ()
Stops the worker, after the actions already queued have run.
- template<typename... Args>
bool [add_action](#) (actionT action, Args &&... args)
Queues an action, bound to args, and says whether it was taken.
- template<typename... Args>
bool [add_task](#) (actionT action, Args &&... args)
Queues a task, and says whether it was taken.
- template<typename otherT>
void [attach](#) (const std::shared_ptr< otherT > &other)
Attaches another execution, so that this one triggers it.
- template<typename otherT>
bool [detach](#) (otherT &other)
Detaches an execution previously attached by [attach\(\)](#).
- auto [results](#) () const
The return values of the actions run by [run\(\)](#), in the order they ran.

Static Public Member Functions

- template<typename... Args>
static std::shared_ptr< execution > [create_instance](#) (Args &&... args)
Creates an execution owned by a std::shared_ptr, which is what binding to it requires.
- template<typename classT, typename T>
static void [bind_action_and_method](#) (actionT &action, const std::shared_ptr< classT > &obj, T classT::*method, const std::shared_ptr< execution > &async_exec)
Binds asynchronously an external action to a class function member.
- template<typename T>
static void [bind_action_and_function](#) (actionT &action, T Fn, const std::shared_ptr< execution > &async_exec)
Binds asynchronously an external action to a plain function.

Public Attributes

- `std::function< void(void)>` [action_execute](#)
Connection point that runs this execution's queued actions. Bound to `execute_actions()`.
- `std::function< void(void)>` [action_stop](#)
Connection point that stops this execution. Bound to `stop()`.
- `std::function< bool(void)>` [action_is_running](#)
Connection point that reports whether this execution is running. Bound to `is_running()`.
- `std::function< void(void)>` [on_finished](#)
Called on the worker's thread once the queue has drained. Assigned by the caller.
- `std::function< void(std::exception_ptr)>` [on_error](#)
Called on the worker's thread with whatever an action threw. Assigned by the caller.

Static Public Attributes

- `static constexpr auto default_name = "default_name"`
Name of an execution built unnamed.

8.2.1 Detailed Description

```
template<typename actionT>
class untangle::async::execution< actionT >
```

Asynchronous template execution class.

An async execution provides a mechanism to queue work and execute it sequentially on a separate thread. It may also attach another [execution](#) object and trigger its actions. This way actions of different types may be executed on the same thread.

Two kinds of work may be queued. An action, through [add_action\(\)](#), is fire and forget: it runs, and the caller hears nothing more about it. A task, through [add_task\(\)](#), carries the callback it must notify, and reports what it produced - or, for an action returning void, merely that it finished. A queued call has no invocation left for the actuator's own callback convention to read a callback from, which is why a task carries its own.

The mechanism relies on an "asynchronous binding" created by [bind_action_and_method\(\)](#) or [bind_action_and_function\(\)](#).

Template Parameters

<code>actionT</code>	It represents the type of the action. It is specified as <code>std::function<...></code> and should match the signature of the bound function or class method.
----------------------	--

8.2.2 Constructor & Destructor Documentation

8.2.2.1 `execution()` [1/2]

```
template<typename actionT>
untangle::async::execution< actionT >::execution () [inline]
```

Constructs an execution with the default name. Call [create_instance\(\)](#) instead.

The name identifies this execution in the warnings it reports, and is otherwise unused.

8.2.2.2 execution() [2/2]

```
template<typename actionT>
untangle::async::execution< actionT >::execution (
    std::string exec_name) [inline], [explicit]
```

Constructs a named execution. Use [create_instance\(\)](#) for one that is to be bound.

Parameters

exec_name	- A name for this execution.
-----------	------------------------------

8.2.2.3 ~execution()

```
template<typename actionT>
untangle::async::execution< actionT >::~~execution () [inline]
```

Destroys the execution, once its worker has left.

The worker is detached and cannot be joined, so this waits on the "running" state instead. It detaches this execution from whatever attached it first, and returns at once for one that was never started.

Attention

Safe only because `running_` is cleared as the very last thing the worker does: this object may be freed the moment it reads false, so nothing may follow it in `execute()` or `loop()`.

Remarks

No rule of five, deliberately. An execution is declared once and used in place; the members already delete the copy operations several times over.

8.2.3 Member Function Documentation

8.2.3.1 create_instance()

```
template<typename actionT>
template<typename... Args>
std::shared_ptr< execution > untangle::async::execution< actionT >::create_instance (
    Args &&... args) [inline], [static]
```

Creates an execution owned by a `std::shared_ptr`, which is what binding to it requires.

An execution built any other way is still usable - [add_action\(\)](#) takes actions directly - but cannot be bound, and the attempt does not compile.

Parameters

args	- Constructor arguments: a name, or nothing for the default name.
------	---

Returns

- A std::shared_ptr owning the new execution.

8.2.3.2 bind_action_and_method()

```
template<typename actionT>
template<typename classT, typename T>
void untangle::async::execution< actionT >::bind_action_and_method (
    actionT & action,
    const std::shared_ptr< classT > & obj,
    T classT::* method,
    const std::shared_ptr< execution< actionT > > & async_exec) [inline], [static]
```

Binds asynchronously an external action to a class function member.

It creates an action as a binding to a class method (by untangle::bind()), and assigns to action a callable that queues it. The execution is held weakly, so the action may outlive it; the action itself is shared with every call queued from it, rather than copied per call.

Attention

Invoking action after the execution has been destroyed throws untangle::invalid_action. The worker catches it and drops the action with a warning.

Parameters

action	[in,out] - An action of type std::function<...>.
obj	- A std::shared_ptr that wraps the bound class object.
method	- Pointer to function member. It is specified as &<class type>::<function member>.
async_exec	- A std::shared_ptr owning the execution the action is queued on. Requiring it here is what keeps an execution that no shared_ptr owns from being bound at all.

8.2.3.3 bind_action_and_function()

```
template<typename actionT>
template<typename T>
void untangle::async::execution< actionT >::bind_action_and_function (
    actionT & action,
    T Fn,
    const std::shared_ptr< execution< actionT > > & async_exec) [inline], [static]
```

~~Binds asynchronously an external action to a plain function.~~

Generated by Doxygen

It wraps Fn in an action and assigns to action a callable that queues it. The execution is held weakly, as in [bind_action_and_method\(\)](#), and the action is shared with every call queued from it rather than copied per call.

Parameters

action	[in,out] - An action of type <code>std::function<...></code> .
Fn	- A plain function, or any callable; moved into the binding.
async_exec	- A <code>std::shared_ptr</code> owning the execution the action is queued on.

8.2.3.4 `is_running()`

```
template<typename actionT>
bool untangle::async::execution< actionT >::is_running () const    [inline]
```

Is a worker of this execution's still there?

True from [run\(\)](#) or [start\(\)](#) until the worker has left, which is the same instant [~execution\(\)](#) waits for and the same one [run\(\)](#) waits for before starting another.

Remarks

[on_finished](#) is raised from inside the drain and reads this as true - the worker is standing in the callback with the rest of its path still to go.

Returns

true - a worker is still there.

8.2.3.5 `is_busy()`

```
template<typename actionT>
bool untangle::async::execution< actionT >::is_busy () const    [inline]
```

Is this execution working through its action queue?

True from the moment an action is added until the last one has returned. Unlike [is_running\(\)](#), which is about the worker thread, this answers "has it room for more work"

- a worker started by [start\(\)](#) is running for its whole life whether busy or idle.

Remarks

Answers for this execution's own actions, not for those it triggers through [attach\(\)](#).

A false answer is durable only for the caller that is itself adding the actions; a true one may stop being true the moment after.

Returns

true - actions are queued, or one is running.

8.2.3.6 run()

```
template<typename actionT>
void untangle::async::execution< actionT >::run () [inline]
```

Runs what is queued on a worker of its own, once, and keeps the results.

The worker is detached: wait for it with `is_running()` or `execution_poll`. Actions queued while it is draining are run by it too. `results()` is filled by this path only.

Blocks until a worker still resident from a previous run has left, so one execution never has two. `is_running()` reads false at exactly that moment, so a caller that waits for it and then runs again does not wait here at all.

Attention

Never call this from inside an action: the worker that would have to leave is the one making the call, so it waits for itself and hangs.

8.2.3.7 start()

```
template<typename actionT>
void untangle::async::execution< actionT >::start () [inline]
```

Starts a continuous worker that runs whatever is queued until `stop()`.

Unlike `run()`, the worker stays for the life of the execution, so `is_running()` is true whether or not there is work; ask `is_busy()` instead. Results are not collected.

Blocks until a worker still resident has left, as `run()` does.

Attention

Never call this from inside an action, for the reason given on `run()`.

8.2.3.8 stop()

```
template<typename actionT>
void untangle::async::execution< actionT >::stop () [inline]
```

Stops the worker, after the actions already queued have run.

Remarks

An action passed to `add_action()` after this call is refused, not queued: `stop()` is the end of this execution's working life, and an action accepted then would never run.

8.2.3.9 add_action()

```
template<typename actionT>
template<typename... Args>
bool untangle::async::execution< actionT >::add_action (
    actionT action,
    Args &&... args) [inline]
```

Queues an action, bound to args, and says whether it was taken.

It runs later, on whichever thread drives this execution - its own worker, or an attacher's. Queueing before [run\(\)](#) or [start\(\)](#) is normal.

Attention

A stopped execution refuses and drops the action, silently for a caller that ignores the answer. Nothing is thrown: [untangle::invalid_action](#) means a dead binding, while a refused action is sound and merely too late.

Remarks

A caller arriving through [bind_action_and_method\(\)](#) or [bind_action_and_function\(\)](#) never sees the answer: those lambdas return `actionT::result_type`, which has no room for it.

It does not notify anyone. The answer below is the last the caller hears: what the action returns is run and dropped, and nothing reports that it finished. Use [add_task\(\)](#) for work that has to report. A trailing callable passed here is an ordinary argument of the action - the actuator's callback convention reads a trailing argument of an invocation, and a queued call has none.

Template Parameters

Args	- The argument types the action is bound to.
------	--

Parameters

action	- The action to queue.
args	- The arguments to bind to action.

Returns

true - queued, and this execution's worker will run it.

false - the execution is stopped and the action was dropped.

8.2.3.10 add_task()

```
template<typename actionT>
template<typename... Args>
bool untangle::async::execution< actionT >::add_task (
    actionT action,
    Args &&... args) [inline]
```

Queues a task, and says whether it was taken.

A task is an action bound to its arguments and to the callback it must notify - unlike [add_action\(\)](#), which queues work the caller hears nothing more about. It runs on whichever thread drives this execution, and the callback is invoked there too, with what the action returned. Queueing before [run\(\)](#) or [start\(\)](#) is normal.

Attention

The last argument is the callback, and this signature cannot say so: a parameter pack cannot be followed by a deducible parameter, so it arrives inside args and [untangle::bind_task\(\)](#) splits it off. It must be callable with the action's result and return nothing - or callable with nothing at all, when the action returns void, because finished is the message and the result is optional. A missing or unusable one is a `static_assert` rather than silence.

Finished does not mean failed. A task whose action throws is not notified - there is no result to hand over - and what it threw goes to [on_error](#), as a failing action's does. A caller waiting on the callback alone would wait for ever.

Remarks

The actions convention on [untangle::actuator](#) does not apply here and never did: it reads a trailing argument of an invocation, and a queued call has no invocation left to read from. That is what a task is for.

Template Parameters

Args	- The argument types to bind, of which the last is the callback.
------	--

Parameters

action	- The action to run.
args	- The arguments to bind to action, followed by the callback to notify.

Returns

true - queued, and this execution's worker will run it and notify the callback.

false - refused and dropped, so it will never run and never notify. Either the execution is stopped, or the task could not do its job: no callback to notify, or nothing to run. The answer is the whole report - nothing is thrown and the callback is not invoked.

8.2.3.11 attach()

```
template<typename actionT>
template<typename otherT>
void untangle::async::execution< actionT >::attach (
    const std::shared_ptr< otherT > & other) [inline]
```

Attaches another execution, so that this one triggers it.

The attached execution's pending actions are run by this one's worker, and stopping this one stops that one too. Actions of different types can be run on one thread this way.

The attachment is a pointer into other; what makes that safe is the record left on the other side, which [~execution\(\)](#) and [detach\(\)](#) use to take it back out.

Parameters

other	- A std::shared_ptr owning the execution to trigger; requiring one keeps an unowned execution from being attached at all. Attaching the same execution twice adds it twice, and one detach() still removes it completely.
-------	---

Exceptions

invalid_attachment	- if other is attached already. An execution has at most one attacher, which keeps the graph a forest and the cycle check a walk rather than a search.
invalid_attachment	- if the attachment would close a cycle, itself included. Driving a cycle recurses until the stack is gone, so it is refused where the caller can still be told.

8.2.3.12 detach()

```
template<typename actionT>
template<typename otherT>
bool untangle::async::execution< actionT >::detach (
    otherT & other) [inline]
```

Detaches an execution previously attached by [attach\(\)](#).

Both paths are unwired together, the triggering one and the stopping one.

Parameters

other	- The execution to stop triggering. Detaching one that was never attached is not an error: the caller asked for a state that already holds.
-------	---

Returns

- true - other was attached and is no longer.
- false - other was not attached to this execution.

8.2.3.13 results()

```
template<typename actionT>
auto untangle::async::execution< actionT >::results () const [inline]
```

The return values of the actions run by [run\(\)](#), in the order they ran.

Read it from [on_finished](#), which fires once the queue has drained, so the results are complete by the time the callback sees them. Returned by value: the worker is appending to it.

Remarks

Only [run\(\)](#) fills this. A continuous worker has no point at which a run is over, so [start\(\)](#) collects nothing.

Returns

The results of the most recent run, or empty if none has produced any.

8.2.4 Member Data Documentation

8.2.4.1 action_execute

```
template<typename actionT>
std::function<void(void)> untangle::async::execution< actionT >::action_execute
```

Connection point that runs this execution's queued actions. Bound to [execute_actions\(\)](#).

Attention

Wired by the constructor. Assigning to it unwires whatever is connected, silently.

8.2.4.2 action_stop

```
template<typename actionT>
std::function<void(void)> untangle::async::execution< actionT >::action_stop
```

Connection point that stops this execution. Bound to [stop\(\)](#).

[attach\(\)](#) wires it together with [action_execute](#), and [detach\(\)](#) unwires both.

Attention

Wired by the constructor. Assigning to it unwires whatever is connected, silently.

8.2.4.3 `action_is_running`

```
template<typename actionT>
std::function<bool(void)> untangle::async::execution< actionT >::action_is_running
```

Connection point that reports whether this execution is running. Bound to [is_running\(\)](#).

[execution_poll](#) holds its address, and [~execution\(\)](#) takes it back out.

Attention

Wired by the constructor. Assigning to it unwires whatever is connected, silently.

8.2.4.4 `on_finished`

```
template<typename actionT>
std::function<void(void)> untangle::async::execution< actionT >::on_finished
```

Called on the worker's thread once the queue has drained. Assigned by the caller.

[results\(\)](#) is complete by the time it runs, and [is_running\(\)](#) reads true there.

Remarks

It is raised once per pass, not once per piece of work, and a pass that ran only tasks raises it like any other. Do not read it as "my task finished": that is the task's own callback, and it has already run by this point, inside the same drain and on this thread.

A worker that woke with an empty queue has finished nothing and stays silent.

8.2.4.5 `on_error`

```
template<typename actionT>
std::function<void(std::exception_ptr)> untangle::async::execution< actionT >::on_error
```

Called on the worker's thread with whatever an action threw. Assigned by the caller.

The action is dropped either way; this is what it is dropped to. Without it the only record is a warning on stderr naming this execution, which reaches a log and no code — an action that failed and one that succeeded are the same from outside. A handler assigned here replaces that warning, so a caller that takes responsibility for reporting is not reported twice.

Remarks

It receives a `std::exception_ptr` because the worker catches everything, including what is not a `std::exception`. Rethrow it to read it.

It is told about a dead binding too, which arrives as `untangle::invalid_action`. A caller asking why an action did not run wants both answers, not the one the header happens to raise itself.

A task's failure arrives here as well, and so does its callback's. A task whose action throws is not notified - there is no result to hand over - so this is the only report it makes. And because a callback runs inside the same try as the task that owns it, this can fire for a task whose action in fact succeeded: the work was done, and only the telling failed.

Attention

It runs on the worker's thread, inside the drain, like [on_finished](#). Nothing may escape it: it is called from the same try the action was called from, and an exception leaving a detached thread function calls `std::terminate`.

The documentation for this class was generated from the following file:

- `async.hpp`

8.3 untangle::async::execution_poll Class Reference

Poll that reports whether any [execution](#) added to it is still running.

```
#include <async.hpp>
```

Public Member Functions

- `execution_poll ()=default`
Constructs an empty poll. `add()` is what puts anything in it.
- `~execution_poll ()`
Destroys the poll, releasing every execution still in it.
- `template<typename asyncexecT>`
`void add (asyncexecT &async_exec)`
Adds an [execution](#) object to the poll.
- `template<typename asyncexecT>`
`void remove (asyncexecT &async_exec)`
Removes an [execution](#) object from the poll.
- `void withdraw (const std::function< bool(void)> *action)`
Takes one action pointer out, without touching the execution it came from.
- `auto is_running ()`
Checks if the execution objects are running.

8.3.1 Detailed Description

Poll that reports whether any [execution](#) added to it is still running.

Remarks

An execution runs in a detached thread and cannot be joined, so checking the "running" state is the only way to wait for one. This checks any number of them at once.

A poll is constructed and owned by the caller, and answers for the executions added to that one. An owner of a set of workers - a thread pool is the case this was written for - waits for its own and not for whatever else the process is running.

8.3.2 Constructor & Destructor Documentation

8.3.2.1 ~execution_poll()

```
untangle::async::execution_poll::~~execution_poll () [inline]
```

Destroys the poll, releasing every execution still in it.

Remarks

The record runs both ways, and it has to: each side holds a pointer into the other, so whichever dies first takes itself out of the other. `~execution()` withdraws from every poll holding it; this tells every execution it holds that this poll is gone. Neither order leaves a dangling pointer, which is what lets a poll be a local.

8.3.3 Member Function Documentation

8.3.3.1 add()

```
template<typename asyncexecT>
void untangle::async::execution_poll::add (
    asyncexecT & async_exec) [inline]
```

Adds an [execution](#) object to the poll.

Parameters

async_exec	An execution object.
------------	--------------------------------------

8.3.3.2 remove()

```
template<typename asyncexecT>
void untangle::async::execution_poll::remove (
    asyncexecT & async_exec) [inline]
```

Removes an [execution](#) object from the poll.

Attention

[add\(\)](#) stores a pointer into the object, so one destroyed while still added would leave the poll calling into freed memory. [~execution\(\)](#) calls this.

Parameters

async_exec	An execution object.
------------	--------------------------------------

8.3.3.3 withdraw()

```
void untangle::async::execution_poll::withdraw (
    const std::function< bool(void)> * action) [inline]
```

Takes one action pointer out, without touching the execution it came from.

What [~execution\(\)](#) calls for each poll holding it: by then the object is being destroyed and must not be called back into, so the two halves of [remove\(\)](#) are separated here.

Parameters

action	- The address add() stored.
--------	---

8.3.3.4 is_running()

```
auto untangle::async::execution_poll::is_running () [inline]
```

Checks if the execution objects are running.

Returns

true - if at least one [execution](#) object in this poll is running.

The documentation for this class was generated from the following file:

- [async.hpp](#)

8.4 untangle::async::invalid_attachment Struct Reference

Raised by [execution::attach\(\)](#) when the attachment asked for is refused.

```
#include <async.hpp>
```

Public Member Functions

- [invalid_attachment](#) (std::string text)
Construct a new invalid attachment object.
- const char * [what](#) () const noexcept override
The message text describing the reason of this exception.

8.4.1 Detailed Description

Raised by [execution::attach\(\)](#) when the attachment asked for is refused.

Remarks

Not an `untangle::invalid_action`: that one reports a binding whose target has died and is swallowed by the worker, while this is a caller error and must not be.

8.4.2 Constructor & Destructor Documentation

8.4.2.1 invalid_attachment()

```
Generated by Doxygen
untangle::async::invalid_attachment::invalid_attachment (
    std::string text) [inline], [explicit]
```

Construct a new invalid attachment object.

Parameters

text	- A message text, describing the reason of this exception.
------	--

8.4.3 Member Function Documentation

8.4.3.1 what()

```
const char * untangle::async::invalid_attachment::what () const    [inline], [override], [noexcept]
```

The message text describing the reason of this exception.

Returns

const char* - The message text.

The documentation for this struct was generated from the following file:

- async.hpp

Chapter 9

File Documentation

9.1 async.hpp

```
00001 // Copyright (c) 2018 Nicolae Popescu. MIT License.
00002
00006 #pragma once
00007
00008 #include <actuator/actuator.hpp>
00009 #include <algorithm>
00010 #include <atomic>
00011 #include <chrono>
00012 #include <condition_variable>
00013 #include <exception>
00014 #include <functional>
00015 #include <iterator>
00016 #include <memory>
00017 #include <mutex>
00018 #include <print>
00019 #include <string>
00020 #include <thread>
00021 #include <type_traits>
00022 #include <utility>
00023 #include <vector>
00024
00025 namespace untangle {
00026 namespace async {
00030
00037 struct invalid_attachment : std::exception {
00043     explicit invalid_attachment(std::string text) : message_(std::move(text)) {}
00044
00050     const char* what() const noexcept override { return message_.c_str(); }
00051
00052 private:
00053     std::string message_;
00054 };
00055
00064 struct attachment {
00066     std::weak_ptr<attachment> attacher;
00067 };
00068
00079 class execution_poll {
00080 public:
00084     execution_poll() = default;
00085
00094     ~execution_poll() {
00095         std::vector<registration> held;
00096
00097         {
00098             std::lock_guard<std::mutex> lock(actuator_mutex_);
00099             held.swap(registrations_);
00100         }
00101
00102         // Outside the lock: each of these takes the execution's own, and ~execution() takes them in
00103         // the opposite order. Releasing this one first is what keeps the two from meeting in a cycle.
00104         for (auto& one : held) {
00105             one.forget();
00106         }
00107     }
00108
00114     template <typename asyncexecT>
```

```

00115 void add(asyncexecT& async_exec) {
00116     {
00117         std::lock_guard<std::mutex> lock(actuator_mutex_);
00118
00119         if (!actuator_is_running_.is_connected()) {
00120             actuator_is_running_ = untangle::connect(async_exec.action_is_running);
00121         } else {
00122             actuator_is_running_.add(&async_exec.action_is_running);
00123         }
00124
00125         registrations_.emplace_back(&async_exec.action_is_running,
00126                                     [this, &async_exec] { async_exec.forget_poll(*this); });
00127     }
00128
00129     // Outside the lock, and deliberately: this takes the execution's own. The execution has to
00130     // know which polls hold a pointer into it, because ~execution() is what takes it back out and
00131     // there is now more than one poll it could be in.
00132     async_exec.remember_poll(*this);
00133 }
00134
00143 template <typename asyncexecT>
00144 void remove(asyncexecT& async_exec) {
00145     withdraw(&async_exec.action_is_running);
00146     async_exec.forget_poll(*this);
00147 }
00148
00157 void withdraw(const std::function<bool(void)>* action) {
00158     std::lock_guard<std::mutex> lock(actuator_mutex_);
00159
00160     actuator_is_running_.remove(action);
00161     std::erase_if(registrations_,
00162                  [action](const registration& one) { return one.action == action; });
00163 }
00164
00170 auto is_running() {
00171     std::lock_guard<std::mutex> lock(actuator_mutex_);
00172     actuator_is_running_();
00173
00174     auto result = false;
00175     for (const auto& ret : actuator_is_running_.results) {
00176         result = result || ret;
00177     }
00178     return result;
00179 }
00180
00181 private:
00186 struct registration {
00187     const std::function<bool(void)>* action;
00188     std::function<void()> forget;
00189 };
00190
00191 actuator<std::function<bool(void)>> actuator_is_running_;
00192 std::vector<registration> registrations_;
00193 mutable std::mutex actuator_mutex_;
00194 };
00195
00215 template <typename actionT>
00216 class execution {
00217     // An execution may attach one of a different specialisation, which is a different class;
00218     // attach() and detach() reach into its attachment_lifetime_.
00219     template <typename otherActionT>
00220     friend class execution;
00221
00222     // add() and remove() keep the record this object holds of the polls that hold it.
00223     friend class execution_poll;
00224
00225 public:
00235     template <typename... Args>
00236     static std::shared_ptr<execution> create_instance(Args&&... args) {
00237         return std::make_shared<execution>(std::forward<Args>(args)...);
00238     }
00239
00245     execution() : execution(std::string(default_name)) {}
00246
00252     explicit execution(std::string exec_name) : name(std::move(exec_name)) {
00253         action_execute = untangle::bind(this, &execution<actionT>::execute_actions);
00254         action_stop = untangle::bind(this, &execution<actionT>::stop);
00255         action_is_running = untangle::bind(this, &execution<actionT>::is_running);
00256     }
00257
00272     ~execution() {
00273         // Out of the attacher first, before the worker is even asked to stop: from here on nothing
00274         // may reach this object, and the actions about to be destroyed are the ones it holds.
00275         if (!attachment_lifetime_->attacher.expired()) {
00276             attacher_execute->remove(&action_execute);
00277             attacher_stop->remove(&action_stop);
00278         }

```

```

00279
00280 {
00281     std::lock_guard<std::mutex> lock(action_mutex_);
00282     started_ = false;
00283     stopped_ = true;
00284 }
00285
00286 action_cv_.notify_all();
00287
00288 while (running_.load()) { // time of check
00289     std::this_thread::sleep_for(std::chrono::milliseconds(1));
00290 }
00291
00292 // time of use: this object is freed once the destructor returns, so the check above is only
00293 // safe because running_ is the last thing the worker touches.
00294 //
00295 // Out of every poll that holds a pointer into this object: an execution can be in several,
00296 // and an address left in any of them outlives the object. Taken by swap so nothing else can be
00297 // added to the record while this walks it, and withdrawn rather than removed because remove()
00298 // would call back into an object that is going away.
00299 std::vector<execution_poll*> holding_polls;
00300
00301 {
00302     std::lock_guard<std::mutex> lock(polls_mutex_);
00303     holding_polls.swap(polls_);
00304 }
00305
00306 for (auto* poll : holding_polls) {
00307     poll->withdraw(&action_is_running);
00308 }
00309 }
00310
00328 template <typename classT, typename T>
00329 static void bind_action_and_method(actionT& action, const std::shared_ptr<classT>& obj,
00330                                     T classT::* method,
00331                                     const std::shared_ptr<execution>& async_exec) {
00332     auto async_action = std::make_shared<const actionT>(untangle::bind(obj, method));
00333
00334     action = [wp = std::weak_ptr<execution>(async_exec),
00335              async_action](auto&&... args) -> actionT::result_type {
00336         // lock() also keeps the execution alive for the duration of the call
00337         const auto exec = wp.lock();
00338         if (!exec) {
00339             throw invalid_action("bind: invalid execution");
00340         }
00341
00342         exec->add_queued_action([async_action, ... args = std::forward<decltype(args)>(args)] {
00343             return (*async_action)(args...);
00344         });
00345
00346         return typename actionT::result_type();
00347     };
00348 }
00349
00361 template <typename T>
00362 static void bind_action_and_function(actionT& action, T Fn,
00363                                     const std::shared_ptr<execution>& async_exec) {
00364     auto async_action = std::make_shared<const actionT>(std::move(Fn));
00365
00366     action = [wp = std::weak_ptr<execution>(async_exec),
00367              async_action](auto&&... args) -> actionT::result_type {
00368         const auto exec = wp.lock();
00369         if (!exec) {
00370             throw invalid_action("bind: invalid execution");
00371         }
00372
00373         exec->add_queued_action([async_action, ... args = std::forward<decltype(args)>(args)] {
00374             return (*async_action)(args...);
00375         });
00376
00377         return typename actionT::result_type();
00378     };
00379 }
00380
00392 bool is_running() const { return running_.load(); }
00393
00409 bool is_busy() const {
00410     std::lock_guard<std::mutex> lock(action_mutex_);
00411     return has_pending_actions_or_tasks() || executing_action_.load();
00412 }
00413
00427 void run() {
00428     wait_thread_to_finish();
00429     collecting_results_ = true;
00430     thread_ = std::thread(&execution::execute, this);
00431     thread_.detach();
00432 }

```

```

00433
00434 void start() {
00435     wait_thread_to_finish();
00436     collecting_results_ = false;
00437     {
00438         std::lock_guard<std::mutex> lock(action_mutex_);
00439         started_ = true;
00440         stopped_ = false;
00441     }
00442     thread_ = std::thread(&execution::loop, this);
00443     thread_.detach();
00444 }
00445
00446 void stop() {
00447     {
00448         std::lock_guard<std::mutex> lock(action_mutex_);
00449         started_ = false;
00450         stopped_ = true;
00451     }
00452
00453     // The worker drains what is already queued before it leaves loop(), so stop() does not have to
00454     // spin on the queue; waking it is enough.
00455     action_cv_.notify_all();
00456
00457     if (actuator_stop_.is_connected()) {
00458         actuator_stop_();
00459     }
00460 }
00461
00462 template <typename... Args>
00463 bool add_action(actionT action, Args&&... args) {
00464     return add_queued_action(std::bind(std::move(action), std::forward<Args>(args)...));
00465 }
00466
00467 template <typename... Args>
00468 bool add_task(actionT action, Args&&... args) {
00469     return add_queued_task(untangle::bind_task(std::move(action), std::forward<Args>(args)...));
00470 }
00471
00472 template <typename otherT>
00473 void attach(const std::shared_ptr<otherT>& other) {
00474     if (static_cast<const void*>(other.get()) == static_cast<const void*>(this)) {
00475         throw invalid_attachment("attach: an execution cannot be attached to itself");
00476     }
00477
00478     if (!other->attachment_lifetime_->attacher.expired()) {
00479         throw invalid_attachment("attach: that execution is attached already");
00480     }
00481
00482     // At most one attacher each makes the graph a forest, so a cycle closes exactly when other is
00483     // somewhere up this chain. Walking it is the whole check.
00484     for (auto link = attachment_lifetime_->attacher.lock(); link; link = link->attacher.lock()) {
00485         if (link == other->attachment_lifetime_) {
00486             throw invalid_attachment("attach: the attachment would close a cycle");
00487         }
00488     }
00489
00490     if (!actuator_execute_.is_connected()) {
00491         actuator_execute_ = untangle::connect(other->action_execute);
00492     } else {
00493         actuator_execute_.add(&other->action_execute);
00494     }
00495
00496     if (!actuator_stop_.is_connected()) {
00497         actuator_stop_ = untangle::connect(other->action_stop);
00498     } else {
00499         actuator_stop_.add(&other->action_stop);
00500     }
00501
00502     // What ~execution() needs to take itself back out of these actuators: the link proves this
00503     // attacher is still there, and the two pointers are only ever followed while it does.
00504     other->attachment_lifetime_->attacher = attachment_lifetime_;
00505     other->attacher_execute_ = &actuator_execute_;
00506     other->attacher_stop_ = &actuator_stop_;
00507 }
00508
00509 template <typename otherT>
00510 bool detach(otherT& other) {
00511     // The actuator's own action list is the record of what is attached; nothing else has to keep
00512     // one. An action is stored as a pointer, so the attachment is found by identity.
00513     const auto& actions = actuator_execute_.actions;
00514     if (std::find(actions.begin(), actions.end(), &other.action_execute) == actions.end()) {
00515         return false;
00516     }
00517
00518     actuator_execute_.remove(&other.action_execute);

```

```

00625     actuator_stop_.remove(&other.action_stop);
00626
00627     other.attachment_lifetime_>attacher.reset();
00628     other.attacher_execute_ = nullptr;
00629     other.attacher_stop_ = nullptr;
00630
00631     return true;
00632 }
00633
00645 auto results() const {
00646     std::lock_guard<std::mutex> lock(results_mutex_);
00647     return results_;
00648 }
00649
00650 static constexpr auto default_name = "default_name";
00651
00657 std::function<void(void)> action_execute;
00658
00666 std::function<void(void)> action_stop;
00667
00676 std::function<bool(void)> action_is_running;
00677
00689 std::function<void(void)> on_finished;
00690
00716 std::function<void(std::exception_ptr)> on_error;
00717
00718 std::string name = default_name;
00719
00720 private:
00721 using queued_action_t = std::function<typename actionT::result_type(void)>;
00722
00727 void remember_poll(execution_poll& poll) {
00728     std::lock_guard<std::mutex> lock(polls_mutex_);
00729
00730     polls_.push_back(&poll);
00731 }
00732
00736 void forget_poll(execution_poll& poll) {
00737     std::lock_guard<std::mutex> lock(polls_mutex_);
00738
00739     std::erase(polls_, &poll);
00740 }
00741
00751 bool has_pending_actions_or_tasks() const {
00752     return action_actuator_.is_connected() || action_actuator_.has_tasks();
00753 }
00754
00764 bool add_queued_action(queued_action_t action) {
00765     {
00766         std::lock_guard<std::mutex> lock(action_mutex_);
00767
00768         // Once stopped, the worker is on its way out and would never reach this action; dropping it
00769         // here is what keeps it from sitting in the queue looking as though it were pending.
00770         if (stopped_) {
00771             std::println(stderr, "warning: execution '{}' is stopped, action not added", name);
00772             return false;
00773         }
00774
00775         action_actuator_.add(std::move(action));
00776     }
00777
00778     action_cv_.notify_one();
00779     return true;
00780 }
00781
00796 bool add_queued_task(untangle::task_t task) {
00797     {
00798         std::lock_guard<std::mutex> lock(action_mutex_);
00799
00800         // As for an action: a stopped worker would never reach it, and a task left in the queue
00801         // would look pending while being unable to ever notify.
00802         if (stopped_) {
00803             std::println(stderr, "warning: execution '{}' is stopped, task not added", name);
00804             return false;
00805         }
00806
00807         if (!action_actuator_.add_task(std::move(task))) {
00808             std::println(stderr, "warning: execution '{}' refused a task that cannot report", name);
00809             return false;
00810         }
00811     }
00812
00813     action_cv_.notify_one();
00814     return true;
00815 }
00816
00826 void execute_actions() {

```

```

00827     actions_and_tasks_run_ = 0;
00828
00829     for (;;) {
00830         // Takes over action_actuator_ under the lock; its actions then run without it.
00831         actuator<queued_action_t> batch;
00832
00833         {
00834             std::lock_guard<std::mutex> lock(action_mutex_);
00835             if (!has_pending_actions_or_tasks()) {
00836                 break;
00837             }
00838
00839             batch = std::move(action_actuator_);
00840
00841             // Under the same lock, so is_busy() sees both or neither.
00842             executing_action_ = true;
00843         }
00844
00845         // Counted before either call: a dead binding leaves the actions list during operator(),
00846         // and call_tasks() consumes the tasks list outright.
00847         const auto in_the_batch = batch.actions.size() + batch.tasks.size();
00848
00849         // The actions have their own arms inside the actuator; this one is for the work around
00850         // them, and for the rule that nothing may leave this loop: an exception escaping a detached
00851         // thread function calls std::terminate.
00852         try {
00853             batch();
00854
00855             // Empty for a void action type, which is what makes this one line rather than two arms.
00856             if (collecting_results_) {
00857                 std::lock_guard<std::mutex> lock(results_mutex_);
00858                 std::ranges::move(batch.results, std::back_inserter(results_));
00859             }
00860
00861             // After the actions, and deliberately: one pass fires every action it holds and then every
00862             // task, so within a batch the two kinds do not interleave in the order they were queued.
00863             // Across batches the queue is still first in, first out. A task reports to its own
00864             // callback, so nothing is collected here; what one throws lands in batch.errors below,
00865             // beside what an action threw.
00866             batch.call_tasks();
00867         } catch (...) {
00868             report_error(std::current_exception());
00869         }
00870
00871         for (const auto& thrown : batch.errors) {
00872             report_error(thrown);
00873         }
00874
00875         actions_and_tasks_run_ += in_the_batch;
00876
00877         executing_action_ = false;
00878
00879         // The notification is raised without the lock: on_finished may call add_action(), which
00880         // takes it.
00881         bool drained = false;
00882         {
00883             std::lock_guard<std::mutex> lock(action_mutex_);
00884             drained = !has_pending_actions_or_tasks();
00885         }
00886
00887         if (drained) {
00888             notify_finished();
00889         }
00890     }
00891
00892     // Last, and part of the pass: an attachment is only ever reached through action_execute, which
00893     // is this function, so whoever drives this execution drives the ones attached to it.
00894     if (actuator_execute_.is_connected()) {
00895         actuator_execute_();
00896     }
00897 }
00898
00904 void report_error(std::exception_ptr thrown) {
00905     if (on_error) {
00906         // The handler is the caller's code, on a detached thread; anything escaping it would leave
00907         // the thread function and call std::terminate.
00908         try {
00909             on_error(thrown);
00910         } catch (...) {
00911             std::println(stderr,
00912                 "warning: execution '{}'" had on_error throw, and dropped what it threw", name);
00913         }
00914
00915         return;
00916     }
00917 }
00918 try {

```

```

00919     std::rethrow_exception(thrown);
00920 } catch (const invalid_action& ia) {
00921     std::println(stderr, "warning: execution '{}' dropped an invalid action: {}", name,
00922         ia.what());
00923 } catch (const std::exception& e) {
00924     std::println(stderr, "warning: execution '{}' dropped an action that threw: {}", name,
00925         e.what());
00926 } catch (...) {
00927     std::println(stderr, "warning: execution '{}' dropped an action that threw an unknown type",
00928         name);
00929 }
00930 }
00931
00932 void notify_finished() {
00933     if (actions_and_tasks_run_ == 0 || !on_finished) {
00934         return;
00935     }
00936     {
00937         // An action may queue another, so a pass that drained can leave more behind it. That is the
00938         // next batch, not the end of this one.
00939         std::lock_guard<std::mutex> lock(action_mutex_);
00940         if (has_pending_actions_or_tasks()) {
00941             return;
00942         }
00943     }
00944     on_finished();
00945 }
00946
00947 void wait_thread_to_finish() {
00948     auto resident = false;
00949     while (!running_.compare_exchange_weak(resident, true)) {
00950         resident = false;
00951         std::this_thread::sleep_for(std::chrono::milliseconds(1));
00952     }
00953 }
00954
00955 void execute() {
00956     // The results belong to this run, not to this run and the last one.
00957     {
00958         std::lock_guard<std::mutex> lock(results_mutex_);
00959         results_.clear();
00960     }
00961     execute_actions();
00962     std::println("execution '{}' finishing thread", name);
00963
00964     // Must stay last: ~execution() may free this object the moment it reads false.
00965     running_ = false;
00966 }
00967
00968 void loop() {
00969     for (;;) {
00970         {
00971             std::unique_lock<std::mutex> lock(action_mutex_);
00972
00973             // Bounded: an attached execution has its own list and cannot notify this condition
00974             // variable.
00975             action_cv_.wait_for(lock, std::chrono::milliseconds(10),
00976                 [this] { return has_pending_actions_or_tasks() || !started_; });
00977
00978             if (!started_ && !has_pending_actions_or_tasks()) {
00979                 break;
00980             }
00981         }
00982         execute_actions();
00983     }
00984 }
00985
00986 // A last pass for the attachments; this execution's own list is already empty here.
00987 execute_actions();
00988
00989 std::println("execution '{}' thread finished", name);
00990
00991 // Must stay last: ~execution() may free this object the moment it reads false.
00992 running_ = false;
00993 }
00994
00995 std::thread thread_;
00996
00997 std::atomic_bool started_ = {false};
00998 std::atomic_bool running_ = {false};
00999
01000 // Set by stop(), and the difference between "not started yet" and "finished for good": actions
01001 // may be queued before run()/start(), but not after stop().
01002 std::atomic_bool stopped_ = {false};

```

```

01020
01021 // action_actuator_, started_ and stopped_ are written by every thread that calls add_action() or
01022 // stop() and read by the worker; nothing touches them outside this mutex.
01023 mutable std::mutex action_mutex_;
01024 std::condition_variable action_cv_;
01025
01031 actuator<queued_action_t> action_actuator_;
01032
01040 using void_actuator = actuator<std::function<void(void)>>;
01041
01042 void_actuator actuator_execute_;
01043 void_actuator actuator_stop_;
01044
01056 std::shared_ptr<attachment> attachment_lifetime_ = std::make_shared<attachment>();
01057
01069 void_actuator* attacher_execute_ = nullptr;
01070 void_actuator* attacher_stop_ = nullptr;
01071
01072 // std::vector cannot hold void type; use an arbitrary type e.g. int
01073 using resultT = std::conditional<std::is_void<typename actionT::result_type>::value, int,
01074                               typename actionT::result_type>;
01075
01079 std::vector<typename resultT::type> results_;
01080
01084 mutable std::mutex results_mutex_;
01085
01092 std::atomic_bool collecting_results_ = {false};
01093
01099 std::atomic_bool executing_action_ = {false};
01100
01102 std::vector<execution_poll*> polls_;
01103 mutable std::mutex polls_mutex_;
01104
01116 std::atomic_size_t actions_and_tasks_run_ = {0};
01117 };
01118 } // namespace async
01119 } // namespace untangle

```

Index

- ~execution
 - untangle::async::execution< actionT >, [22](#)
- ~execution_poll
 - untangle::async::execution_poll, [31](#)
- action_execute
 - untangle::async::execution< actionT >, [29](#)
- action_is_running
 - untangle::async::execution< actionT >, [29](#)
- action_stop
 - untangle::async::execution< actionT >, [29](#)
- add
 - untangle::async::execution_poll, [32](#)
- add_action
 - untangle::async::execution< actionT >, [25](#)
- add_task
 - untangle::async::execution< actionT >, [26](#)
- async, [1](#)
- attach
 - untangle::async::execution< actionT >, [27](#)
- bind_action_and_function
 - untangle::async::execution< actionT >, [23](#)
- bind_action_and_method
 - untangle::async::execution< actionT >, [23](#)
- create_instance
 - untangle::async::execution< actionT >, [22](#)
- detach
 - untangle::async::execution< actionT >, [28](#)
- execution
 - untangle::async::execution< actionT >, [21](#)
- invalid_attachment
 - untangle::async::invalid_attachment, [33](#)
- is_busy
 - untangle::async::execution< actionT >, [24](#)
- is_running
 - untangle::async::execution< actionT >, [24](#)
 - untangle::async::execution_poll, [33](#)
- namespace untangle: functions, [15](#)
- on_error
 - untangle::async::execution< actionT >, [30](#)
- on_finished
 - untangle::async::execution< actionT >, [30](#)
- remove
 - untangle::async::execution_poll, [32](#)
- results
 - untangle::async::execution< actionT >, [28](#)
- run
 - untangle::async::execution< actionT >, [24](#)
- start
 - untangle::async::execution< actionT >, [25](#)
- stop
 - untangle::async::execution< actionT >, [25](#)
- untangle, [17](#)
- untangle::async::attachment, [19](#)
- untangle::async::execution< actionT >, [19](#)
 - ~execution, [22](#)
 - action_execute, [29](#)
 - action_is_running, [29](#)
 - action_stop, [29](#)
 - add_action, [25](#)
 - add_task, [26](#)
 - attach, [27](#)
 - bind_action_and_function, [23](#)
 - bind_action_and_method, [23](#)
 - create_instance, [22](#)
 - detach, [28](#)
 - execution, [21](#)
 - is_busy, [24](#)
 - is_running, [24](#)
 - on_error, [30](#)
 - on_finished, [30](#)
 - results, [28](#)
 - run, [24](#)
 - start, [25](#)
 - stop, [25](#)
- untangle::async::execution_poll, [31](#)
 - ~execution_poll, [31](#)
 - add, [32](#)
 - is_running, [33](#)
 - remove, [32](#)
 - withdraw, [32](#)
- untangle::async::invalid_attachment, [33](#)
 - invalid_attachment, [33](#)
 - what, [34](#)
- what
 - untangle::async::invalid_attachment, [34](#)
- withdraw
 - untangle::async::execution_poll, [32](#)